

## Slide 4:

Python:

Saída esperada: -8 8 9

Java:

Saída esperada: 3345

JavaScript:

Saída esperada: 3 52

C:

Saída esperada: 0

Python:

Saída esperada: padrão True

JavaScript:

Saída esperada: true false true

## Slide 5:

Expressões aritméticas: questões de projeto:

Quais são as regras de precedência?

A precedência de operadores determina a ordem em que diferentes operadores são executados em uma expressão sem parênteses. Operadores com maior precedência são avaliados primeiro

Quais são as regras de associatividade?

A associatividade determina a ordem de execução quando dois ou mais operadores possuem a mesma precedência em uma expressão

Em que ordem os operandos são avaliados?

A ordem de avaliação dos operandos (os valores em si) indica qual lado de um operador é calculado primeiro. Na maioria das linguagens modernas (como Java, C# e Python), a avaliação ocorre estritamente da esquerda para a direita

Há restrições a efeitos colaterais?

Sim, e misturar efeitos colaterais (como modificar uma variável enquanto ela é lida) na mesma expressão pode gerar comportamentos imprevisíveis

O usuário pode sobrecarregar operadores?

Depende estritamente da linguagem de programação

Que mistura de tipos é permitida?

A mistura de tipos depende se a linguagem possui tipagem forte ou fraca, e estática ou dinâmica

## Slide 22:

JavaScript:

Código do slide: (printa total:0012)

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

Código do corrigido:

```
const precos = [10, 20, 30];
let total = 0;
for (const p of precos)
    total += p;
console.log("Total: " + total);
```

Principal erro: o código está percorrendo o array no lugar de cada elemento do array

Python:

Código do slide:(O principal erro nesse código é um erro de lógica ao tentar passar o valor zero para o parâmetro d)

```
def desconto(preco, d=None):  
    d = d or 10      # padrão: 10%  
    return preco * (100 - d) / 100  
  
print(desconto(200, 0))
```

Código do corrigido:

```
def desconto(preco, d):  
    if d is None:  
        d=0  
    return preco * (100 - d) / 100  
  
print(desconto(200, None))
```

Principal erro: ele sempre pegava o valor de d como None mesmo que fosse passado um valor para ele e aplica o valor de 10% de desconto

## Slide 23:

Java:

Código do slide: (sempre printa “inválido”)

```
public class Main {  
    public static void main(String[] args) {  
        int dia = 2; String nome = "";  
        switch (dia) {  
            case 1: nome = "domingo";  
            case 2: nome = "segunda";  
            case 3: nome = "terça";  
            default: nome = "inválido";  
        }  
        System.out.println(nome);  
    }  
}
```

Código corrigido: (printa o dia certo)

```
public class Main {
```

```

public static void main(String[] args) {
    int dia = 2; String nome = "";
    switch (dia) {
        case 1: nome = "domingo";
        break;
        case 2: nome = "segunda";
        break;
        case 3: nome = "terça";
        break;
        default: nome = "inválido";
        break;
    }
    System.out.println(nome);
}
}

```

Principal erro: falta de “break;” o código só rodava todos os cases sem parar

C:

Código do slide: (não possuía nenhum comando se o if for verdadeiro)

```

#include <stdio.h>
#include <stdlib.h>

int main(){
    int saldo = 100, saque = 50;
    if (saque > saldo);
    printf("saldo insuficiente\n");
    printf("fim\n");

    return 0;
}

```

Código do corrigido: (printa se o saldo é inválido ou não)

```

#include <stdio.h>
#include <stdlib.h>

int main(){
    int saldo = 100, saque = 50;

```

```

    if (saque > saldo){
        printf("saldo insuficiente\n");
    }
    printf("fim\n");

    return 0;
}

```

Principal erro: O programa sempre imprime a mensagem porque há um ponto e vírgula logo após o fechamento dos parênteses do if

## Slide 24:

Go:

Código do slide: (o novo valor não é atribuído por que o operador usado no código é o :=)

```

package main
import "fmt"

func main() {
    x := 10
    if x > 5 {
        x := x * 2
        fmt.Println("dentro:", x)
    }
    fmt.Println("fora:", x)
}

```

Código corrigido:

```

package main
import "fmt"

func main() {
    x := 10
    if x > 5 {
        x = x * 2
        fmt.Println("dentro:", x)
    }
}

```

```
}  
    fmt.Println("fora:", x)  
}
```

Principal erro: O operador usado é o `:=` e ele não altera valores já aplicado a uma variável, o operador correto para ser usado seria o `=`

Código do slide: (a divisão inteira será truncada e imprimirá o valor errado)

```
public class Main {  
    public static void main(String[] args) {  
        int acertos = 7, total = 10;  
        double taxa = acertos / total * 100;  
        System.out.println(taxa + "%");  
    }  
}
```

Código corrigido:

```
public class Main {  
    public static void main(String[] args) {  
        int acertos = 7, total = 10;  
        double taxa = (acertos * 100.0) / total;  
        System.out.println(taxa + "%");  
    }  
}
```

Principal erro: A divisão foi feita com números inteiros no lugar de valores decimais como float, por isso precisa primeiro multiplicar por 100 depois dividir pelo total